

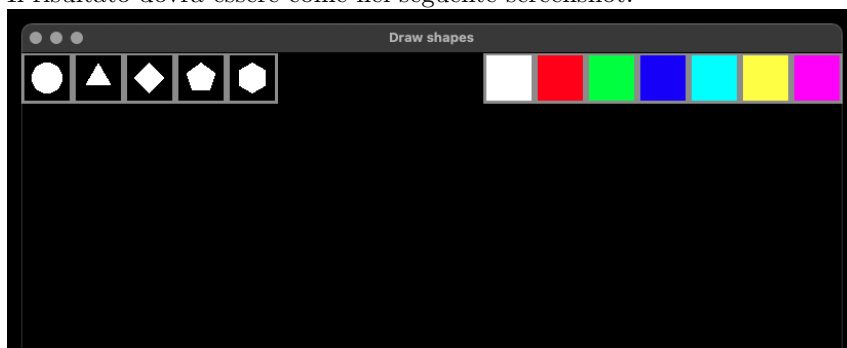
Esercitazione 20/3/25

Disegno interattivo di forme

1. Partiamo dal template `my_drawing.cpp`
2. Organizziamo l'area della finestra in due rettangoli:
 - una *menubar* alta 50 pixel nella parte superiore
 - una *canvas* riservata al disegno nell'area restante

Per farlo è necessario aggiungere informazioni allo stato, che ci permettano di verificare in che area si trova il mouse: in questo caso, è sufficiente memorizzare il valore delle *y* che separa i due rettangoli.

3. Prevediamo di utilizzare:
 - cinque forme, alle quali assegniamo convenzionalmente i valori da 0 a 4: cerchio, triangolo, quadrato, pentagono e esagono; per disegnarle useremo sempre la primitiva `sf::CircleShape` con un numero diverso di punti: 30 se l'indice è 0, $2+i$ per indice $i>0$;
 - sette colori, ai quali assegniamo convenzionalmente i valori da 0 a 6: bianco, rosso, verde, blu, ciano, giallo, magenta; conviene memorizzare, nello stato o come costante di programma, un array di `sf::Color` che mantiene nelle rispettive posizioni i valori simbolici di questi colori.
4. Nella funzione `doGUI`, disegnamo un menu con 5 bottoni quadrati con lato 50 pixel, interno trasparente e bordo grigio interno di 3 pixel, accostato a sinistra.
5. All'interno di questi bottoni disegnamo, in ordine da sinistra a destra, le cinque forme specificate sopra. Tutte queste forme devono essere di colore bianco, centrate rispetto al bottone e non in contatto col bordo del bottone.
6. Disegnamo un altro menu con 5 bottoni quadrati con lato 50 pixel, accostato a destra; l'interno di questi bottoni andrà colorato con i sette colori specificati sopra e avrà bordo grigio interno di 3 pixel.
7. Il risultato dovrà essere come nel seguente screenshot:



8. Rendiamo i bottoni clickabili:
 - aggiungiamo allo stato due membri `active_shape` e `active_color`

- che memorizzano l'indice della shape e del colore attivi nei due menu; inizializziamo entrambi questi membri a 0 (cerchio bianco)
- modifichiamo la `doGui` in modo che disegni di un colore diverso il bordo dei bottoni corrispondenti alla shape e al colore attivi
 - nella handle che gestisce l'evento `MouseButtonPressed`, verificare se l'evento avviene quando il cursore del mouse si trova in un menu; in tal caso, cambiare il valore di shape/colore attivo secondo l'area del bottone in cui si trovava il mouse quando si è verificato l'evento.
9. Disegniamo shape col mouse (tasto sinistro):
- definiamo una struct per memorizzare i parametri di una shape generica disegnata sulla canvas: tipo (0-4); colore (0-6); grandezza (per ora fissa a 30); posizione (un `sf::Vector2f`)
 - aggiungiamo allo stato un vector `shapes` di tali struct, per memorizzare le shape disegnate, inizialmente vuoto
 - nella handle che gestisce l'evento `MouseButtonPressed`, verifichiamo se l'evento avviene quando il cursore del mouse si trova nella canvas e proviene dal bottone sinistro (useremo il destro per altri scopi); in tal caso, generiamo una nuova shape del tipo e colore attivi e posizionata dove si trova il mouse e aggiungere tale shape al vector di cui sopra
 - nella `doGraphics` cicliamo sul contenuto di tale vettore e disegniamo le shape nella canvas.
10. Spostiamo le shape disegnate col mouse (tasto destro):
- aggiungiamo allo stato un membro `selected_shape` per memorizzare la shape selezionata per l'editing, nell'elenco di quelle contenute in `shapes`; inizializziamo tale membro a -1 (nessuna shape selezionata)
 - nella handle che gestisce l'evento `MouseButtonPressed`, verifichiamo se l'evento avviene quando il cursore del mouse si trova nella canvas e proviene dal bottone destro; in questo caso, verifichiamo se il click è avvenuto quando il mouse si trovava su una shape disegnata; a questo scopo, cicliamo sul contenuto di `shapes` e per ciascuna vediamo se la posizione del mouse dista dalla sua posizione meno della sua grandezza; la distanza si può calcolare con la funzione di utility `dist`; se ne troviamo una, variamo il valore di `selected_shape` di conseguenza
 - nella handle che gestisce l'evento `MouseButtonReleased` riportiamo il valore di `selected_shape` a -1. Pertanto, una shape sarà selezionata solo fintantoché il bottone destro del mouse resta premuto.
 - gestione del drag: nella handle che gestisce l'evento `MouseMove`, se c'è una shape selezionata, cambiamo la sua posizione aggiungendole un offset pari al movimento del mouse rispetto al frame precedente; a questo scopo, aggiungiamo allo stato un membro `mouse_pos` che ricorda la posizione al frame precedente: `mouse_pos` deve essere inizializzato alla pressione del tasto destro e aggiornato a ogni movimento del mouse.
 - mettiamo in evidenza la shape selezionata, modificando la `doGraphics`: se l'indice della shape da disegnare coincide con

`selected_shape`, disegniamo il suo bordo in grigio chiaro, altrimenti non disegniamo il bordo.

11. Cancelliamo la shape selezionata:
 - nelle handle che gestisce l'evento `TextEntered`, se viene premuto il tasto backspace `'\b'` mentre c'è una shape selezionata (tasto destro del mouse premuto), eliminiamo tale shape da `shapes` e ripristiamo `selected_shape` al valore -1. È possibile, in alternativa, gestire la cancellazione nella handle dell'evento `KeyPressed`; in questo caso, il controllo sul codice del tasto premuto va fatto diversamente (consultare la classe `Keyboard` per i codici dei tasti).